

MIRROR: Local Evidence Bundles for Reproducible Research Software Review, with a JANUS Case Study

Anton Sokolov

Researcher, Tyche Institute, Tallinn, Estonia

ORCID: 0000-0003-2452-7096

30 May 2026

Abstract

MIRROR is a small research-software method for keeping drafts, scripts, source copies, and claim records reviewable while an iterative research project is still moving. A MIRROR bundle places an artifact, structured claims, provenance notes, source descriptors or retained source copies, assumptions, and SHA-256 digests into a local directory that can be verified offline. This article reports the design as an Issues in Research Software experience paper and uses JANUS, a multilingual drift-triage manuscript and data packet, as the worked case. JANUS is not a second main contribution; it is a deliberately messy case in which claims, generated outputs, source boundaries, public-safe examples, scripts, and reviewer risk had to be separated before release. The central claim is narrow: byte-level replay and schema conformance can reduce the cost of later review, but they do not establish truth, completeness, authorship, legal compliance, public release authority, external timestamping, or regulated assurance. The current MIRROR implementation passed 115 local tests and positive and negative verifier fixtures, while the JANUS public-safe packet passed its own build, test, checksum, and leakage checks. The main contribution is not another assurance badge, but a boundary discipline: a reviewer should be able to ask what bytes were retained, which claim points to which evidence, what failed, and what still requires human judgment.

Keywords: research software; reproducibility; provenance; evidence bundles; artifact review; software sustainability

1 Introduction

Research software often fails quietly before it fails technically. A script still runs, but the draft that cites it has changed. A source was once available, but the retained copy is missing. A table is regenerated, but the claim it supports remains stronger than the evidence. In small projects this is annoying; in AI-governance and evidence-infrastructure work it becomes a methodological risk, because language such as “verified”, “attested”, or “compliant” can drift faster than the underlying evidence.

MIRROR started from that practical problem. It is not a new cryptographic primitive and not a replacement for repositories, archives, Research Object Crate (RO-Crate), PROV, software citation, or artifact evaluation practices (1-7). It is a deliberately modest layer that can be used before a project is ready for public release: gather the artifact, the retained evidence, the claims, and the assumptions into one replayable directory; then let an offline verifier answer only the question it is qualified to answer. Did the retained bytes and declared structure survive?

2 Method

MIRROR models a reviewable artifact as a small tuple:

$$B = (a, S, C, N, M),$$

where a is the artifact being checked, S is the set of retained sources or source descriptors, C is a structured claim register, N is a human-readable note file, and M is a manifest containing schema version, hashes, assumptions, and non-signing metadata. The reference verifier implements the narrow predicate:

$$V(B) = \text{ok} \iff \text{schema}(M, C) \wedge H(a) = M_a \wedge \forall s \in S : H(s) = M_s \wedge \text{policy}(C, N),$$

where H is SHA-256 and policy is a local guardrail over configured wording and pointer rules.

Lemma 1: Byte replay is not claim truth. If $V(B) = \text{ok}$, then the retained local files match the recorded hashes and schemas. It does not follow that the research claim is correct, that the retained source is sufficient, or that a public release is safe. The lemma is almost embarrassingly simple, which is why it is useful. It stops the verifier from becoming a badge with better typography.¹

MIRROR supports byte and schema replay, not truth, authorship, legal compliance, external timestamping, or regulated assurance.

The implementation is intentionally ordinary Python. The repository contains a bundler, an offline verifier, schema files, daily roll-up tooling, a signed-transition profile generator that does not sign, and positive and negative fixtures. The test suite covers schema validity, hash mismatch, missing files, unsafe assurance wording, roll-up payload semantics, local anchor-chain validation, reviewer-report packets, and signed-transition profile payloads. A test run on 30 May 2026 completed 115 tests successfully.

Layer	What MIRROR records	What it refuses to infer
Bundle	Artifact bytes, retained sources, claims, notes, assumptions, and digests.	Truth, novelty, fairness of interpretation, or completeness of evidence.
Verifier	Schema conformance, digest equality, required entries, and configured policy failures.	Authorship, peer review, legal compliance, public release readiness, or regulated assurance.
Roll-up	Local artifact-index rows, digest observations, diagnostic categories, and a local Merkle root.	Lane health scores, publication decisions, audits of sibling projects, or external time.
Anchor chain	A hash-linked JSONL sequence over retained local roll-up records.	Notarization, priority proof, certified timestamping, or identity proof.

Table 1: Interpretation boundaries for the current MIRROR workflow. The table is intentionally negative as well as positive: the refusal column is part of the method.

3 Worked Case: JANUS

Here JANUS names the laboratory case-study packet used to exercise MIRROR: a multilingual drift-triage manuscript and data release for policy-adjacent research claims. It was selected because the public packet contains a 19-page manuscript, a source archive, a public data-and-code supplement, aggregate CSVs, plots, curated public edge-suite examples, scoring and plotting scripts, dependency metadata, final checks, and a release manifest. The private workspace also contains raw source claims, generated forward translations, generated back-translations, per-record reports, and operational notes that are not public-safe. That mixture makes JANUS useful for MIRROR: the problem is not merely whether a file can be hashed, but how to keep a claim, the evidence for it, and the public boundary separate.

3.1 Case setup

The JANUS artifact set used in this paper consisted of four public-safe packet files: the clean PDF, the source ZIP, the public data/code supplement ZIP, and the packet status note. The packet also retained source descriptors rather than private source copies: a SHA-256 manifest, the JANUS final-packet manifest, the row-level leakage scan, the supplement file manifest, and the clean-preprint status note. The public supplement contained 21 files: seven CSV files, six Python scripts, five PNG plots, two Markdown casebook files, and one `requirements.txt`. The scan reported public PASS for scanned CSV, text, and script surfaces; binary plots were marked as skipped by the content scanner; reviewer-only bundle material retained expected warnings and stayed out of the public release set.

The case therefore exercised the part of MIRROR that matters most for research-software review: retaining enough evidence to replay the public packet, while refusing to publish raw claims or generated text that could be mistaken for authoritative translations.

3.2 MIRROR procedure applied to JANUS

The MIRROR pass treated the JANUS packet as a case artifact rather than as a proof target. It recorded the packet files and hashes, mapped manuscript claims to public-safe descriptors, checked that scripts and data were available for download, and separated replay evidence from scholarly judgment. The concrete actions were:

1. bundle the JANUS clean-preprint packet as the artifact under review;
2. retain source descriptors for the manuscript, supplement, leakage scan, and hashes;
3. hash retained bytes with SHA-256 and verify them against the packet manifest;
4. map claims about data/code availability, public examples, and redaction boundaries to the packet files;

5. run local tests and leakage scans;
6. record positive outcomes, expected warnings, and unresolved release decisions;
7. keep private raw claims, generated translations, back-translations, and operational notes outside the public case packet.

This is where MIRROR is most useful. It did not decide whether JANUS is a good multilingual method. It made the review surface more explicit: which public files exist, which claims cite them, what the verifier can replay, and which materials remain withheld.

3.3 JANUS findings

The final JANUS packet recorded four public-safe case artifacts and five source-descriptor records. The public supplement exposed 21 downloadable files and the source archive exposed the LaTeX source, bibliography, compiled bibliography, local style file, public edge-case table source, and the aggregate figure used in the paper. The JANUS test pass reported 49 unit tests. The PDF build had 19 pages, no undefined citations or references in the final log scan, and a checksum match against the clean-preprint SHA-256 file. The leakage scan found no public-tier findings in scanned CSV, text, or script surfaces. Four reviewer-only files retained expected warnings and were therefore not promoted to the public tier.

The MIRROR pass changed the manuscript and packet in three practical ways. First, the data-accessibility statement now names the public data/code supplement instead of describing it only abstractly. Second, the public supplement includes scripts and a dependency manifest, not only aggregate outputs. Third, self-referential public examples were replaced with domain-shaped edge examples so the case study shows translation-fragility surfaces rather than its own release machinery.

The JANUS case shows that MIRROR can make review evidence more inspectable, but it does not make JANUS true, complete, legally compliant, peer reviewed, or externally attested.

4 Edge Cases and Negative Results

The JANUS pass made five edge cases visible enough to discuss without exposing private text.

Stale or changed digest. A clean packet is only clean for a specific byte sequence. If the PDF, source ZIP, or supplement ZIP changes after the manifest is written, MIRROR can flag the mismatch. It cannot decide whether the change is harmless; it can only force the author to refresh the packet or narrow the claim.

Ambiguous claim-to-source mapping. A statement such as “the public supplement contains scripts” is easy to support. A stronger statement such as “the supplement fully reproduces the study” is not supported by public JANUS materials, because private claims and generated translations remain withheld. MIRROR helps by mapping the weaker claim to public files and leaving the stronger claim unsupported.

Private source excluded from public package. The JANUS private corpus is intentionally absent. That absence is not a verifier failure. It is a release decision that must be visible, because public reproducibility is partial by design.

Verifier passes bytes but cannot validate truth. A byte-perfect public casebook can still be a poor example, a misleading excerpt, or an overclaimed result. MIRROR can preserve the file and the decision trail; it cannot validate the interpretation.

Evidence package too complete to publish. A package that includes raw claims, generated translations, row identifiers, and operational notes may be excellent for internal review and unsafe for public release. The JANUS split therefore treats reviewer-only warnings as expected outcomes rather than defects to hide.

Daily roll-ups add a related maintenance layer. They scan configured sibling indexes, normalize rows, compute local roots, and append anchor records whose `external_timestamp` value remains null. That null is not a bug. It is a boundary marker: the local chain supports replay of retained local records, not an independent time claim. This makes MIRROR adjacent to content provenance systems such as C2PA, software supply-chain attestations such as SLSA and in-toto, and signing ecosystems such as Sigstore, but it does not claim their trust model or public-log semantics (8-11).

The same distinction matters vertically. MIRROR uses SHA-256 as a conservative byte-integrity primitive under the Secure Hash Standard, not as an oracle about scholarly truth (12). Public-key infrastructure (PKI), the X.509 certificate profile, the European eIDAS trust-services regulation, and European Telecommunications Standards

Institute (ETSI) AdES validation procedures live higher in the governance stack (13-15). MIRROR prepares material that such systems could later sign, seal, time-stamp, or reject; it does not borrow their authority in advance.³

Adjacent practice	Shared concern	MIRROR distinction
PROV and RO-Crate	Describe provenance relationships and package research objects.	Uses a smaller file-system-native packet for claim review before public packaging.
FAIR and FAIR4RS	Make research outputs findable, accessible, interoperable, and reusable.	Focuses on local challengeability before the output is ready to be released.
Software citation	Make software a citable research product.	Records claim-evidence links around the software, not only software identity.
Artifact badging	Evaluate availability, functionality, reuse, or reproduced results.	Produces no badge and deliberately avoids quality grading.
Attestation systems	Bind artifacts to build steps, identities, logs, or provenance predicates.	Keeps v0.1 unsigned and offline; future signing is a separate operator-controlled layer.

Table 2: MIRROR is deliberately near existing provenance and software-sustainability work, but its unit of review is a claim-bearing local research artifact.

5 Lessons for Research Software Practice

The first lesson is that negative semantics need first-class space. Many tools describe what a successful check means; fewer force the author to write what it does not mean. In MIRROR, every green result travels with a non-certification boundary. This is not decorative caution. It prevents a local maintenance check from being confused with a publication decision, conformity assessment, or legal opinion.

The second lesson is that claim registers are more useful than large undifferentiated supplements. A reviewer does not need every file at once. The better question is: which claim is being inspected, what evidence pointer supports it, what is the risk level, and what safe wording would survive if the evidence is weaker than hoped? This is close in spirit to artifact evaluation, but it moves the review unit from “the whole artifact works” to “this statement has a replay path” (7).

The third lesson is that failures should stay boring. A hash mismatch may be important, but it may also mean that a worker edited a file and forgot to refresh the bundle. MIRROR therefore emits deterministic errors, not moral adjectives. The human reviewer still has to decide whether the mismatch is harmless maintenance debt, a publication blocker, or a sign that the claim no longer belongs in public text.

The fourth lesson is that local evidence packages help precisely because they are weaker than public attestations. A private workspace cannot always publish every source, copy every page, or expose every intermediate file. MIRROR can still improve the situation by recording retained bytes, descriptors, assumptions, and blocked-publication notes. That is less glamorous than a public cryptographic log, but it is available at the moment when many mistakes are still cheap to fix.

The fifth lesson is that future signing should be additive. The repository already includes a signed-transition profile generator that hashes an unsigned bundle and a reviewer-report packet, then writes a profile for later operator-controlled signing. It performs no signing itself. This keeps Agent Trust Framework (EATF)-style attestation layers separate from the local replay substrate (17).²

6 Limitations and Threat Model

MIRROR assumes that the local filesystem, retained files, and verifier code are available for inspection. It does not defend against a malicious operating system, hidden key compromise, forged source authority, social pressure, selective omission of claims, or a reviewer who does not read the evidence. It also does not solve licensing. A file may be stable and still not be redistributable; a source descriptor may be honest and still be too weak for a public claim.

The current case is a self-experiment. That is both a strength and a limitation. It gives unusually direct access to iteration history and design choices, but it does not show how independent teams would use the method, how reviewers would score its usefulness, or how it behaves across public repositories with different licensing and contribution cultures. The public-release surface now needs to be treated as part of the method: repository metadata, license text,

citation metadata, reviewer instructions, release notes, and a minimal evidence package are not administrative extras; they are the first reproducibility claim the paper makes.

The security claims are also narrow. SHA-256 digests support byte identity. JSON Schema supports structure. Local policy checks catch configured patterns of unsafe wording. None of these establishes that a claim is right. The best interpretation is maintenance-oriented: MIRROR helps make the next human review sharper, faster, and more reproducible.

7 Conclusion

MIRROR is a small answer to a common research-software problem: evidence moves, claims harden, and later reviewers inherit a pile of plausible files. The method binds artifacts, sources, claims, notes, and digests into an offline packet, then insists on a modest interpretation of the result. That modesty is the point. A useful verifier should help a reviewer challenge a claim without pretending to decide the claim.

Data Accessibility

The MIRROR software, tests, fixtures, paper source, and minimal public-safe evidence package are archived on Zenodo under the Apache License 2.0: <https://doi.org/10.5281/zenodo.20463358> (16). The minimal evidence package is included as `release/mirror-methodology-rc1/` inside the archived `v0.1.1` package.

The JANUS case is reported as a public-safe worked example. Private workspace paths, unpublished third-party material, raw source claims, generated translations, back-translations, per-record reports, and operational notes were excluded from the public package. If the JANUS packet is later deposited separately, its DOI should be added here rather than replaced by a personal repository URL.

Ethics and Consent

The case study uses the author’s own local research workspace and software artifacts. It does not report human-subjects data, private third-party data, customer data, or employer-confidential material. The public version preserves that boundary by excluding private paths, unpublished third-party source material, and any text that implies operational access to external systems.

Funding Information

No external funding is claimed for this preprint.

Competing Interests

The author declares no competing interests for this preprint. The author has professional experience in public-key infrastructure and digital identity systems; no employer, client, or non-public operational material is named or used in this article.

Author Contributions

Anton Sokolov designed the MIRROR workflow, implemented or supervised the local reference tooling, ran the local tests and verifier checks, selected the venue route, and prepared this manuscript.

AI Assistance Disclosure

Author used generative OpenAI Codex / GPT5.5 for final draft polishing. Final responsibility rests with the author, who verified all claims, results, design decisions, and references. Reported per COPE and ICMJE recommendations.

Notes

1. A hash is wonderfully stubborn about bytes and wonderfully silent about meaning. That is a feature only if the paper says so plainly.
2. EATF means Agent Trust Framework: an evidence and agent-attestation framework that can later sign or publish a handoff payload. The current MIRROR workflow does not claim that such signing has occurred.
3. PKI here means public-key infrastructure: the people, policies, certificates, keys, revocation records, software, and institutional habits that make a signature mean more than “some private key produced these bytes”. The cryptography is hard; the governance is where it quietly becomes interesting.

References

1. Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*. 2016;3:160018. DOI: <https://doi.org/10.1038/sdata.2016.18>.
2. Barker M, Chue Hong NP, Katz DS, Lamprecht AL, Martinez C, Psomopoulos F, et al. Introducing the FAIR Principles for research software. *Scientific Data*. 2022;9:622. DOI: <https://doi.org/10.1038/s41597-022-01710-x>.
3. Smith AM, Katz DS, Niemeyer KE, FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*. 2016;2:e86. DOI: <https://doi.org/10.7717/peerj-cs.86>.
4. Groth P, Moreau L, editors. PROV-Overview: An Overview of the PROV Family of Documents. W3C Working Group Note; 2013 Apr 30. Available from: <https://www.w3.org/TR/prov-overview/>.
5. Sefton P, Carragain EO, Soiland-Reyes S, Corcho O, Garijo D, Palma R, et al. RO-Crate Metadata Specification 1.1. researchobject.org community; 2023 Apr 26. DOI: <https://doi.org/10.5281/zenodo.7867028>.
6. Mesirov JP. Accessible reproducible research. *Science*. 2010;327(5964):415-416. DOI: <https://doi.org/10.1126/science.1179653>.
7. Association for Computing Machinery. Artifact Review and Badging Version 1.1. ACM Publications Policy; 2020 Aug 24. Available from: <https://www.acm.org/publications/policies/artifact-review-and-badging-current>.
8. Supply-chain Levels for Software Artifacts. SLSA specification version 1.2. Linux Foundation; cited 2026 May 30. Available from: <https://slsa.dev/spec/v1.2/>.
9. Torres-Arias S, Afzali H, Kuppusamy TK, Curtmola R, Cappos J. in-toto: Providing farm-to-table guarantees for bits and bytes. In: *Proceedings of the 28th USENIX Security Symposium*; 2019; Santa Clara, CA. Available from: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
10. Coalition for Content Provenance and Authenticity. C2PA Specifications 2.1. Joint Development Foundation; cited 2026 May 30. Available from: <https://spec.c2pa.org/specifications/specifications/2.1/index.html>.
11. Sigstore. Overview. OpenSSF / Linux Foundation; cited 2026 May 30. Available from: <https://docs.sigstore.dev/>.
12. National Institute of Standards and Technology. Secure Hash Standard (SHS), FIPS PUB 180-4. Gaithersburg, MD: NIST; 2015 Aug. DOI: <https://doi.org/10.6028/NIST.FIPS.180-4>.
13. Cooper D, Santesson S, Farrell S, Boeyen S, Housley R, Polk W. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280. IETF; 2008 May. DOI: <https://doi.org/10.17487/RFC5280>.
14. European Parliament and Council. Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC. *Official Journal of the European Union*. 2014;L257:73-114. Available from: <https://eur-lex.europa.eu/eli/reg/2014/910/oj/eng>.
15. ETSI. ETSI EN 319 102-1 V1.4.1: Electronic Signatures and Infrastructures (ESI); Procedures for Creation and Validation of AdES Digital Signatures; Part 1: Creation and Validation. Sophia Antipolis: ETSI; 2024 Jun. Available from: https://www.etsi.org/deliver/etsi_en/319100_319199/31910201/01.04.01_60/en_31910201v010401p.pdf.
16. Sokolov A. MIRROR: Local Evidence Bundles for Reproducible Research Software Review. Zenodo; 2026. DOI: <https://doi.org/10.5281/zenodo.20463358>.
17. EATF. Documentation: Agent Trust Framework. EATF; cited 2026 May 30. Available from: <https://www.eatf.eu/docs>.